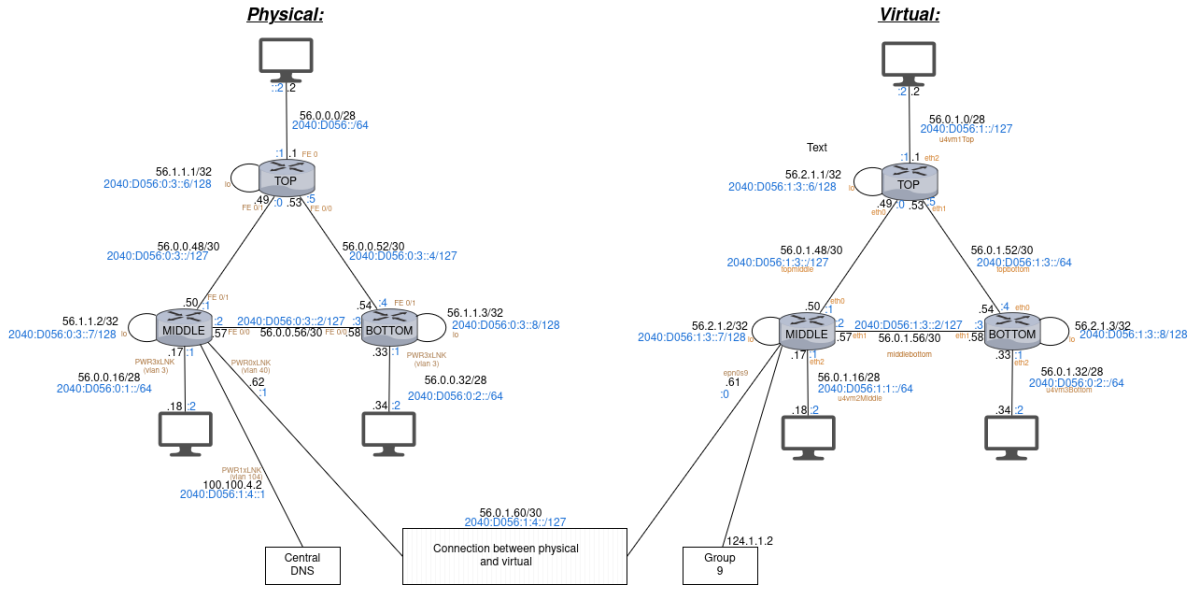
Group 4 Networking Report:

Introduction

The aim of this project was to construct a network using three laptops, three routers, a switch and a server. Then we had to connect the network to other groups and allow them to communicate. Below is the document outlining our process and steps to accomplish this.

General IP Addressing:



IPv6 IPv4

IPv4 Network:

One of the key considerations was how do we define IP addresses between Virtual and Physical configurations. These need to be different to ensure you can correctly access the appropriate device. For this, we opted to denote the separation between virtual and physical sides of the network using:

0 – Physical

1 – Virtual

Why?

Address allocation:

Initially, when setting up the network, we started on the physical side, hence the initial collection of networks starting from 56.0.0.0 (our allotted IPv4 domain). From here, we worked from the outside-in, assigning domains and IP addresses to peripherals first. With the need for all routers to be able to connect to a potential 10 devices, we had to ensure that the assigned regions could contain at least 10 different IP addresses (hence the use of /28 as the net mask, as the remaining 4 bits allow for the assigning of the smallest number of addresses -16 - that will include our specified 10). For every new domain, we had to consider the side of the assigned address block (denoted by the netmask). While we would only be using up to 10 addresses, we need to ensure there is no cross over in domains, and so you start each count after the assigned block (e.g. from 56.0.0.0/28, you are assigning 16 addresses, so the next address needs to be 56.0.0.16). From here, we assigned the IP domains for routers. These were assigned /30 because this allows for the required 2 addresses to form the connection between the 2 routers. The assignment of these follows a similar process.

Separation between Virtual and Physical:

To make assignments easier to remember, we opted to denote virtual addresses with 56.0.1.* and physical addresses with 56.0.0.*. This is because it ensures there is no collision between the domains. The issue with this method is that it results in wasted addresses within our address block that are not assigned to anything. Specifically, it forms a gap between 56.0.0.60 [final address addressable from 56.0.0.56/30] - 56.0.1.0 [first assigned virtual address] which leaves 196 unused addresses.

Static Routes:

Before implementing IPv6 and dynamic routing, we created static routes between routers and set up default gateways on the end devices. When creating static routes on the physical network, we defined both direct and back-up routes between routers. This increased the reliability of the network by providing alternative paths to routers if a direct link ever became ineffective.

However, during the setup we encountered a problem which prevented us from communicating with one of our router's laptops. We spent many hours troubleshooting, involving:

- Comparing running configurations (using '*show run'*)
- Comparing the configuration of the affected laptop with the other laptop using VLAN

- Checking the assigned IP address of the laptop (in */etc/network/interfaces*)
- \Checking simple assignment of a VLAN to the port used
- Ensuring all other attributes such as '*no shutdown'* and spelling were correct

Despite these checks, we couldn't locate the issue and neither router nor its connected laptop could see each other in the network. It turned out that the physical port we were connecting on was broken. The port fa0/1/0 was faulty and so the laptop, nor router, could acknowledge each other's existence. To rectify this issue, we moved the port to fa0/1/1. In hindsight, a simple fix, but in practice we felt like we were losing our minds.

Reasoning for VLANs

Since routers 2 and 3 in the stack (in future will be referred to as Middle and Bottom) were composed of separate boards containing 4 ports. These ports are part of a managed switch, which require segmentation to be performed to operate. Not only does this allow for better organisation and management of the network but also allows you to more easily debug of a more logical network.

IPv6 Diagram:

In a similar way to IPv4, distinctions were made between the virtual and physical networks using the physical representation with a 0, and virtual with a 1. This once again made remembering and switching between the two relatively easy to remember. As an additional positive, if you remembered the IP address for a device on one of the sides, you also knew the device for the other, this made debugging easier and made testing more efficient. This resulted in the following:

Physical – 2040:D056:0::

Virtual – 2040:D056:1::

IS-IS:

Applied on both the virtual and physical sides of our network, this is how we determined our roots dynamically. Both sides required a unique Area ID. This is required to be a unique ID for each region. In our case we chose to use:

49.0001.0000.0000.000*.00 for physical

49.0001.1000.0000.000*.00 for virtual

Where the * denotes the router number in the stack, once again following the convention that 1 is the top and 3 is the bottom.

Virtual:

This required the initiation of an initial isisd.conf file. This would be run on start-up and would enable the use of IS-IS on all the ports it was enabled on in this file. For all router-to-router connections, this was enabled and given the name ‘CORE’.

Physical:

Here, you also need to denote the use of IS-IS on all appropriate interfaces. The intefaces requiring this are only those that point to other routers as this is where the routing information is required.

To allow for the interconnection of routers, the wires between them all must be cross-over cables. These are designed to connect similar devices rather than their inter-device connection counterpart (straight through cables). This was annoying; many of the cables were not labelled and there seemed to be a short supply of crossover cables. These are different as they have the transmit and receive cables swapped over, allowing for 2 of the same device to connect without an intermediary device.

Evidence to support this can be found in the running configuration section for the physical side.

Connecting Physical and Virtual:

Since we started this section on the virtual side, we opted to use the next available addresses from the virtual side (56.0.1.60/30). This also requires for each side of the network to be configures as different AS’s. Since we were given 4050 and 4051, we decided to apply the same logic to assigning AS numbers and signifying the IP addresses, and so the virtual side was designated the 4051 AS number, and the physical was given 4050.

Since a router on the physical side needs to be directly connected to the central switch (plugged into port 14), we opted to configure the link on middle router. This was

because this was the first available port when looking at the stack of switches. Since (as previously established), pwr3xlnk was not functioning, and pwr2xlnk was used for our laptop, we chose to use the next port (pwr1xlnk).

To do this, we had to configure a VLAN with appropriate name “VLAN104”. Since this was originating from the virtual side, the physical interface was assigned the second address in the prefix, 56.0.1.62.

Similarly, on the virtual side, an interface needed to be configured to connect to the physical side. Once again, for uniformity, we chose to connect via the middle router on the virtual side. The next available connection was given this responsibility (eth3).

Since this link was to an external AS, it required the configuration of an eBGP neighbour on the virtual side. This required a specific name and format to allow of the exchange of data between them, to act as if they were one. These configurations were beyond our control; however, policies could be put over this route.

This route was defined in the lab.ext file.

Interdomain Routing:

Following the connection to our own AS, we needed to form a connection to a peer. For this we chose our peering group based on convenience. Since we were behind most teams by the point of suggesting peering, we wanted to peer with a group who seemed at a similar point, as well as were in the room when we were configuring this. We chose to peer with group 9 (also known as group 7, depending on what section of documentation you are reading). This made it difficult to initiate the connection as initially, we formed the connection “g4g7 enp0s8.47”. This, we later found, was incorrect, and so we later changed this to “g4g9 enp0s8.49”, which can be found in the lab.ext file.

Reasoning and decision making

Since the other groups would also need a physical connection to connect their virtual and physical sides, we believed it best to utilise this fact and reduce clutter by connecting to our peers on the virtual side. This would not only make it easier to connect but also reduce the number of ports we’d need. Since we found out that some of our physical ports, were not functional, we wanted to recuse the potential points of error.

Future

We did not set up any policies on this connection however would have liked to add local preference to this link when routing to their IP addresses. This is because this would

simulate the real-world connections that exist between a customer and provider, and how you'd prefer to route to your neighbours IP via this connection rather than routing up through your provider over a connection with has a greater cost.

DNS:

Local DNS

For this, we opted to host it on the laptop connected to the top router (Laptop 1). The reasoning behind this was that we intended to later add on the other services (e.g. Email and TFTP), hosting them on different devices.

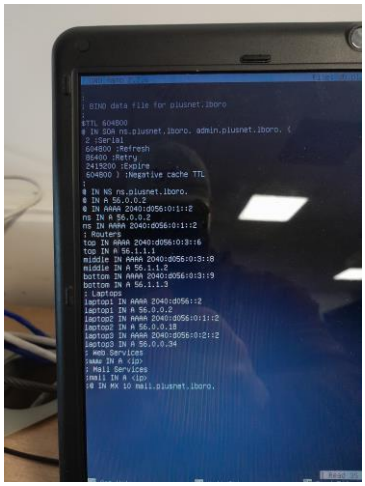
Within our DNS database, we provided access to addresses for each of the physical devices (routers and ports). Not only this, but given the failure of locating an address, it also pointed to the central DNS.

Not only this, but all devices on the network needed to find our DNS. Inside of '/etc' each physical laptop needed to know of the existence of the local DNS. The forwarder for each device was updated to now contain the IP address of the local DNS (56.0.0.2).

DNS configuration

Db.plusnet.lboro

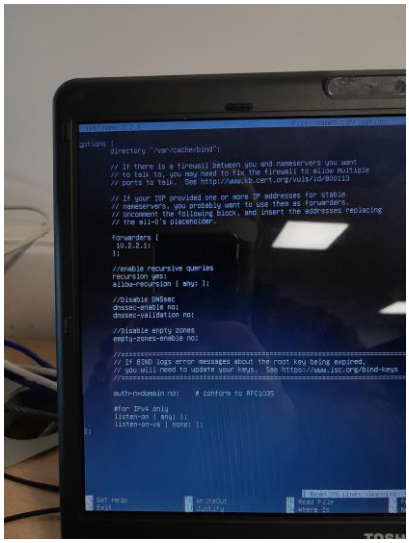
This contains all of the DNS resolutions, going from a name to an actual IP address. This allowed us to be able to get both the IPv4 and IPv6 addresses for devices on our network by name alone.



You can also see our intended links for web and email commented out in the file.

Named.conf.options

This file outlines how the DNS behaves. One of the key parts is the forwarding, where, if not known, the DNS will look at the destination 10.2.2.1 for the next level DNS.



Central DNS

This was hosted on the central server and would be contacted if our local DNS did not hold the requested domain. This connection required a separate VLAN for connection. Once again, for consistency, we used the middle physical router for this connection. This was done on the next available port (pwr0xlnk), where VLAN 104 was connected and assigned the address 100.100.4.2 (the designated address for our groups DNS).

Additional developments

Given more time, we would have also liked to add the addresses of out virtual devices as this is where we would be more likely to add additional resources which exist as this has a greater flexibility and can more easily be re-configured without the need for additional hardware in a more cloud based approach.

BGP:

We encountered many issues while configuring BGP on the virtual side. For example, when we first attempted to configure IPv6 eBGP peers in vtysh, for reasons that were unclear, Quagga failed to start the bgpd.conf file. Attempting to restart the service 'service quagga restart' had no effect, and running ‘show ip bgp summary’on the router returned no output.

We then tried hard-coding the BGP configuration directly in bgpd.conf, but possibly due to formatting issues with the hard-coded configuration, Quagga was still unable to pull up BGP. As a result, we had to rewrite the whole BGP configuration. This time, we only attempted to configure IPv4 BGP. On the router, we declared the remote AS 4050 and specified the prefixes of it. This time, eBGP on both sides successfully exchanged their routing tables, which was a positive sign.

After that, we were able to configure iBGP neighbours within the virtual AS, allowing the remaining routers to learn the physical AS routing table via eBGP. We then specified the gateway and interfaces for u4vm on each eth interface, enabling the virtual PCs to connect to the AS. Eventually, all three virtual u4vms were able to successfully ping the physical laptop, and vice versa, indicating that BGP had been configured successfully.

Due to the IPv6 issues described earlier, we only implemented IPv4 BGP. In theory, by replacing all IPv4 addresses in the BGP configuration with the IPv6 addresses shown in the diagram, BGP should be able to successfully advertise the u4vm addresses over IPv6 as well.

Another issue we encountered was that we were unable to keep our work persistent. We tried the following approaches:

1.Editing bgpd.conf, isisd.conf, and zebra.conf on all three routers using nano and saving the files with Ctrl + O.

2.Running write memory on all three routers.

However, once kathara lclean was executed, all configurations were lost.

We also attempted to copy the above configuration files to the shared folder, but the configurations still could not be preserved.

Our Files and Images:

Virtual:

Lab.conf

This contains all the connections between the virtual devices. We aimed for the naming conventions to be self-explanatory for easy debugging. “U4vm1” for example stands for ‘Unit 4 virtual machine 1’. This lines up with the first virtual laptop/host device you look at when reading the network diagram (form top left to bottom right), another way to think about it is:

On the physical side, laptop1 is connected to the top router in the stack. We tried to carry this over to the virtual side for uniformity.

```
# ------------------------------------
# Router Interconnections (Triangle)
# ------------------------------------

# top ↔ middle
top[0]=net_top_mid
middle[0]=net_top_mid

# middle ↔ bottom
middle[1]=net_mid_bot
bottom[1]=net_mid_bot

# bottom ↔ top
bottom[0]=net_bot_top
top[1]=net_bot_top

# ------------------------------------
# VM Attachments
# ------------------------------------

# top ↔ u4vm1
top[2]=net_top_vm
u4vm1[0]=net_top_vm

# middle ↔ u4vm2
middle[2]=net_mid_vm
u4vm2[0]=net_mid_vm
```

```
# bottom ↔ u4vm3
bottom[2]=net_bot_vm
u4vm3[0]=net_bot_vm

# ------------------------------------
# Higher-level server
# ------------------------------------

middle[3]=g4toPR
middle[4]=g4g9
```

Lab.ext

This file contains all the AS links in the network. Defining their name and link

```
g4g9 enp0s8.49
g4toPR enp0s9
```

Latop.Startup

This contained the assigned virtual IP address for the laptop as well as the interface on the laptop that was use connect to the router. In this snippet, on the IPv6 address has been assigned to u4vm1.

```
ip -6 address add 2040:d056:1::2/64 dev eth0
ip link set eth0 up
ip -6 route add default via 2040:d056:1::1
```

Config File Differences

Since routers Top and Bottom on the virtual sides are very similar in terms of connections and set up, with the only differences being the IPs that are on each interface, they will both be discussed under the same heading in the below section.

bgpd.conf

Routers ‘Top’ and ‘Bottom’ on the virtual side were configured very similarly. This is because, each only connected to two other routers and 1 laptop within the same AS.

```
router bgp 4051
 bgp router-id 56.2.1.1
 network 56.0.1.0/28
 network 2040:d056:1::/64
 neighbor 56.2.1.2 remote-as 4051
 neighbor 2040:d056:1:3::7 remote-as 4051
 neighbor 56.2.1.3 remote-as 4051
 neighbor 2040:d056:1:3::8 remote-as 4051
!
 address-family ipv6
  neighbor 2040:d056:1:3::7 activate
  neighbor 2040:d056:1:3::8 activate
 exit-address-family
```

^top/etc/quagga/bgpd.conf

These files (for both routers) contained the IPv4 and IPv6 addresses of the neighbours within the same AS. It also contains the activation of the IPv6 Addresses, allowing IPv6 addresses to be used in the routing.

The middle router this is different. Since it has connections both within the network (iBGP connections) and outside (eBGP – to our physical AS, as well as group 7), the file is different.

```
router bgp 4051
 bgp router-id 56.2.1.2
 network 56.0.1.16/28
 network 2040:d056:1:1::/64
!
!
!physical connection
 neighbor 56.0.1.62 remote-as 4050
 neighbor 56.0.1.62 description group 4 physical
!
!
!iBGP connection
 neighbor 56.2.1.1 remote-as 4051
 neighbor 2040:d056:1:3::6 remote-as 4051
 neighbor 56.2.1.3 remote-as 4051
 neighbor 2040:d056:1:3::8 remote-as 4051
!
!
 address-family ipv6 unicast
  neighbor 2040:d056:1:4::1 remote-as 4050
  neighbor 2040:d056:1:4::1 activate
  neighbor 2040:d056:1:4::1 description group 4 physical
  neighbor 2040:d056:1:3::6 activate
  neighbor 2040:d056:1:3::8 activate
 exit-address-family
!
!
!eBGP - other group connections
 neighbor 124.1.1.2 remote-as 9081
 neighbor 124.1.1.2 description group 9 virtual
!
end
```

Although minimal in scope, this configuration is fundamental for establishing connectivity between BGP peers.

Config file Consistencies

isisd.conf

This is the same for all routers. While the middle router contains 2 additional interfaces (eth3 and eth4), they do not take part in this IS-IS routing. This is because IS-IS is used to determine the routes available within a network (not to be confused with those from outside the AS – such as those form peers). This results in the sharing and finding of routes to and from every router enabling routes to be determined dynamically.

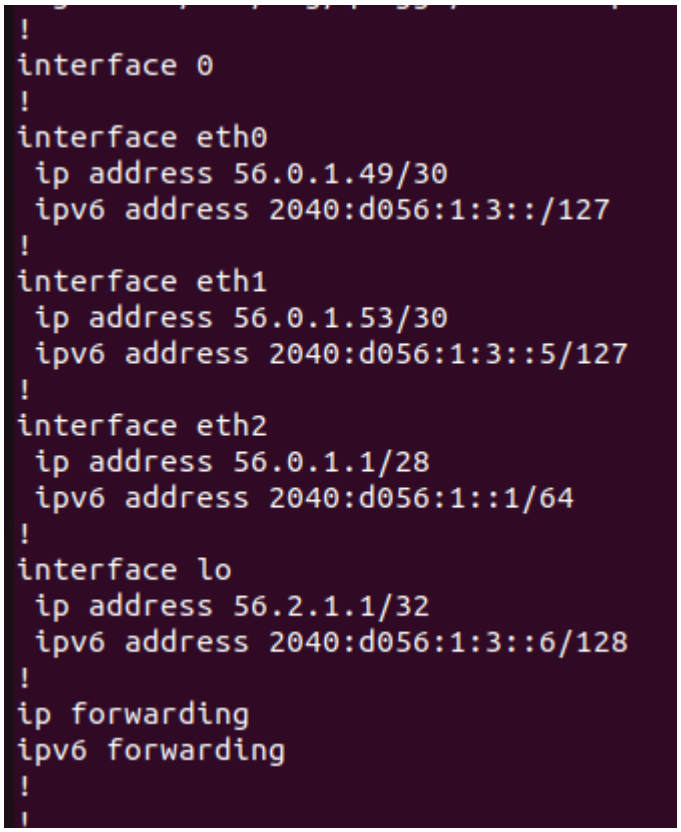
```
!
router isis CORE
 net 49.0001.1000.0000.0001.00
 is-type level-1
 metric-style wide
!
interface eth0
 ip router isis CORE
 ipv6 router isis CORE
!
interface eth1
 ip router isis CORE
 ipv6 router isis CORE
!
interface eth2
 ip router isis CORE
 ipv6 router isis CORE
!
interface lo
 ip router isis CORE
 ipv6 router isis CORE
!
```

^top/etc/quagga/isisd.conf

Here, each interface that is used is taking part in our IS-IS dynamic routing, where “CORE” is just the name that has been applied to this IS-IS instance/tag.

zebra.conf

Another similarity between all the routers is the zebra.conf file. This file outlines the addresses on interfaces.



```
!
interface 0
!
interface eth0
 ip address 56.0.1.49/30
 ipv6 address 2040:d056:1:3::/127
!
interface eth1
 ip address 56.0.1.53/30
 ipv6 address 2040:d056:1:3::5/127
!
interface eth2
 ip address 56.0.1.1/28
 ipv6 address 2040:d056:1::1/64
!
interface lo
 ip address 56.2.1.1/32
 ipv6 address 2040:d056:1:3::6/128
!
ip forwarding
ipv6 forwarding
!
!
```

^top/etc/quagga/zebra.conf

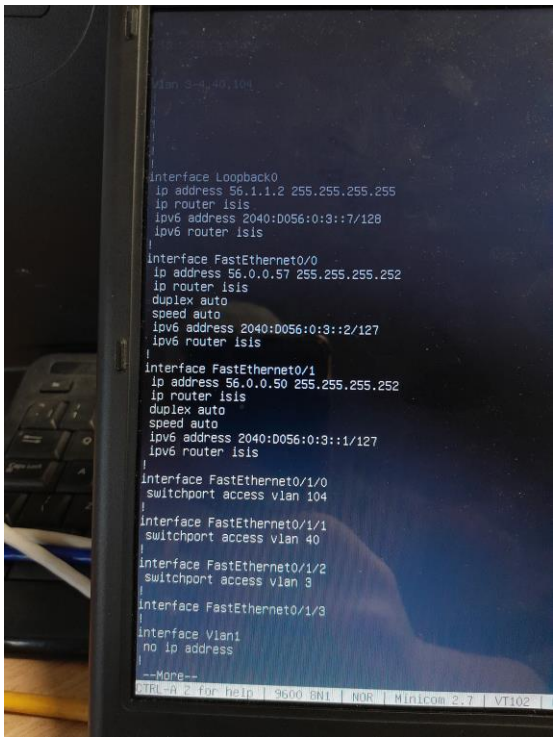
Once again, the Middle routers version of this file consists of additional interfaces (eth3 and 4), the principle is the same. This is where the IP addresses (both in v4 and v6) are assigned to all interfaces.

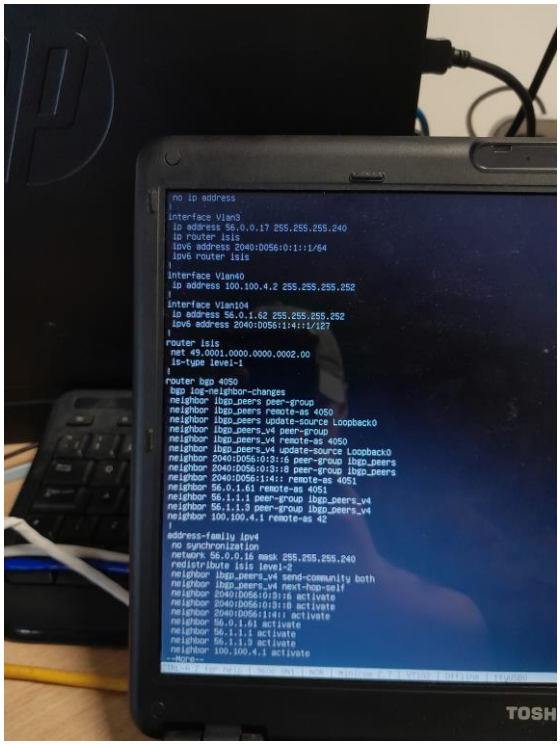
Physical

Show Run:

‘show run’ or ‘show running configuration’

One of our most run commands for both conformation of completeness as well as debugging. This game us an overview of the system and allowed us to find things such as incorrect IP addresses and lack of IS-IS initiation, which helped us resolve issues.





^middle router running configuration

This contains, the interfaces and assigned IP’s as well as the enabling of IPv6 addressing. You can also see that there is no mention of ‘shutdown’. This would have otherwise disabled this interface. You can also see that the neighbouring devices have been configured, in both IPv4 and IPv4, which will allow for iBGP in both v4 and v6. Additionally, you can also see that the eBGP neighbours have also been added, allowing connection to our virtual side and our peer group.

Reflections:

Given more time, we would have liked to not only include the TFTP on laptop3 (physical), so we can backup configurations of the routers in the event of failure or attack, and would have liked to set up an email and a web server.